

BIRZEIT UNIVERSITY

Faculty of Engineering and Technology

Department of Electrical and Computer Engineering

Machine Learning and Data Science - ENCS5341

Image Classification Using Machine Learning

Assignment No.3

Prepared by:

Noura Khdour	1212072
Aseel Dar Assi	1211974
Enas Qutit	1210236

Instructor: Dr. Yazan Abu Farha

Section: 2

Date: 13/1/2026

Birzeit
January -2026

1- Overview

In this project, we solve a weather image classification task, where each image is labeled as Sunny, Cloudy, Rainy, Snowy, or Not Clear. We tested three model types: K-NN as a simple baseline, SVM as a strong traditional model for classification, and CNN as a deep learning model designed for images that can learn useful visual features. We evaluated the models using accuracy and macro F1-score, we also used precision, recall, and a confusion matrix to better understand model errors, especially because the dataset is imbalanced.

2- Data Cleaning and preprocessing

2.1- Extract the ZIP file

In this step, we extract a compressed dataset and automatically selects the most recent file from each folder based on file modification time in case if the folder contains more than file. Only one file per folder is retained and copied to a unified output directory, ensuring data consistency and eliminating outdated files.

2.2- Data Integration and Standardization

Next step, we standardize and merges heterogeneous data files by normalizing column names, resolving encoding and format inconsistencies. The result is a single CSV dataset, ensuring uniform structure and data readiness for further analysis.

```
COLUMN_MAP = {
    'image url': 'image_url',
    'image_url': 'image_url',
    'image link': 'image_url',
    'url': 'image_url',
    'image': 'image_url',

    'desc': 'description',
    'description': 'description',
    'image description': 'description',
    'caption': 'description',
```

Figure 1:A sample of normalizing data columns process.

2.3- Remove duplicates URL links

Then, check if we have a duplicate. Duplicate records were removed using the image URL as a unique identifier. Overall number of unique links is1017.

2.4- Weather Label Standardization

After that, Weather labels were normalized into five standardized categories to eliminate inconsistencies caused by spelling variations, formatting differences, and missing values.

```
'Cloudy': 'Cloudy',
'cloudy': 'Cloudy',
' Cloudy': 'Cloudy',
'cloudy ': 'Cloudy',
'Cloudy ': 'Cloudy',
' Partly Cloudy': 'Cloudy',
'Partly Cloudy': 'Cloudy',
'Partly Cloudy / Sunny': 'Cloudy',
'Cloudy Sunset': 'Cloudy',
'Cloudt': 'Cloudy',
'Windy': 'Cloudy',
'windy': 'Cloudy',
'Cold': 'Cloudy',
```

Figure 2:A sample of normalizing Weather label process

2.5- Image Download and Categorized Storage

In this step, we created five weather category files, then images were downloaded from their URLs and saved into weather-specific directories. **One of the problems** we have faced that some URLs were failed to download because they hosted on restricted platforms (e.g., Google Drive, Pinterest, Pixel etc.) could not be retrieved programmatically and were manually downloaded and categorized accordingly.

2.6- Image Preprocessing and Dataset Preparation

Images were resized, normalized, and converted to a uniform RGB format before being saved as NumPy arrays and organized by weather category for efficient model training.

Note: The final dataset contains **no missing** values across all attributes.

3- Exploratory Data Analysis (EDA)

3.1- Outlier Detection Using Image Brightness

Outliers were detected using the IQR method based on average image brightness. Where lower outliers indicate image is much darker than most images of that weather type (Mean ≈ 0) and Upper outliers indicates image is much brighter than most images of that weather type (Mean ≈ 1). We found that Cloudy have 6 outliers and Sunny have 14 outliers images. To handle the outliers, I just remove them.

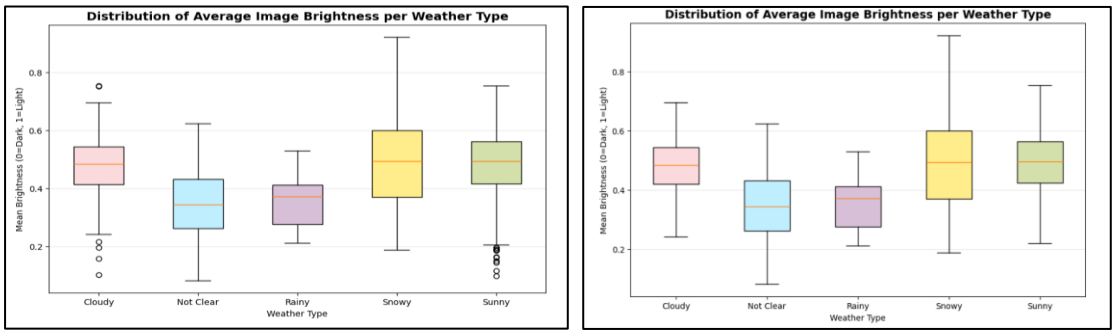


Figure 3:Outliers Detection and Handling using Box plot

3.2- Quantitative Image Statistics

1.Mean: The average brightness of all pixels in the images. Higher mean = brighter images.
2.Median: The middle brightness value when all pixels are sorted. Shows typical brightness without being affected by very dark or very bright spots.
3.Standard Deviation: How much the pixel brightness varies. Higher = more contrast or differences in light and dark areas.
4.Variance: Another way to measure how spread out the pixel brightness is (similar to standard deviation).
5.Skewness: Shows if most pixels are darker or brighter than the average. Positive = more bright pixels, negative = more dark pixels.
6.Kurtosis: Shows if pixel brightness is mostly near the average (Positive =peaked) or spread out (Negative=flat).

7.Entropy: How much "information" or detail the image has. Higher = more complex images with lots of variation.
8.Mode: The brightness value that appears most often in the image.

```
------------------------------
CLOUDY
------------------------------

1- MEAN: 0.4780
2- MEDIAN: 0.4627
3- STANDARD DEVIATION: 0.2652
4- SKEWNESS: 0.1196
5- ENTROPY: 7.9622
6- VARIANCE: 0.0703
8- KURTOSIS: -1.0585
```

```
------------------------------
NOT CLEAR
------------------------------

1- MEAN: 0.3478
2- MEDIAN: 0.2941
3- STANDARD DEVIATION: 0.2530
4- SKEWNESS: 0.6410
5- ENTROPY: 7.7804
6- VARIANCE: 0.0640
8- KURTOSIS: -0.5675
```

```
------------------------------
RAINY
------------------------------

1- MEAN: 0.3555
2- MEDIAN: 0.3098
3- STANDARD DEVIATION: 0.2450
4- SKEWNESS: 0.5986
5- ENTROPY: 7.7762
6- VARIANCE: 0.0600
8- KURTOSIS: -0.5266
```

```
------------------------------
SNOWY
------------------------------

1- MEAN: 0.4960
2- MEDIAN: 0.4824
3- STANDARD DEVIATION: 0.2760
4- SKEWNESS: 0.0512
5- ENTROPY: 7.9763
6- VARIANCE: 0.0762
8- KURTOSIS: -1.1344
```

```
------------------------------
SUNNY
------------------------------

1- MEAN: 0.4907
2- MEDIAN: 0.4863
3- STANDARD DEVIATION: 0.2709
4- SKEWNESS: 0.0261
5- ENTROPY: 7.9764
6- VARIANCE: 0.0734
8- KURTOSIS: -1.0751
```

Figure 4:Quantitative Image Statistics

3.3- Class Distribution of Weather Dataset.

The dataset we have suffers from class imbalance, as seen in Fig 5 the number of images differs across weather categories. This imbalance may bias the learning process toward majority classes and negatively affect classification performance on underrepresented categories.

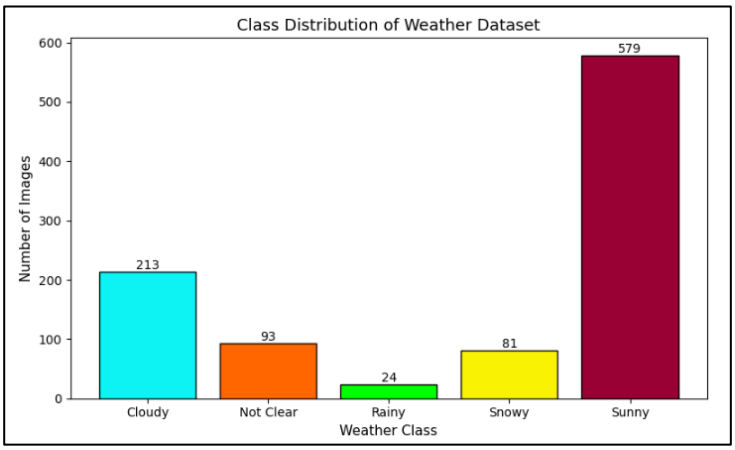


Figure 5:Bar plot

3.4- Pixel Brightness Skewness Analysis

We analyzed the pixel brightness distribution of images for each weather category. Skewness was calculated to understand whether images are generally darker, brighter, or balanced. Positive skew indicates darker images, negative skew indicates brighter images, and values near zero show balanced brightness. The results show that **Sunny**, **Snowy and Cloudy** images have more balanced brightness while, **Rainy** and **Not Clear** images are darker. This analysis helps understand lighting differences across weather categories supports better preprocessing and model design.

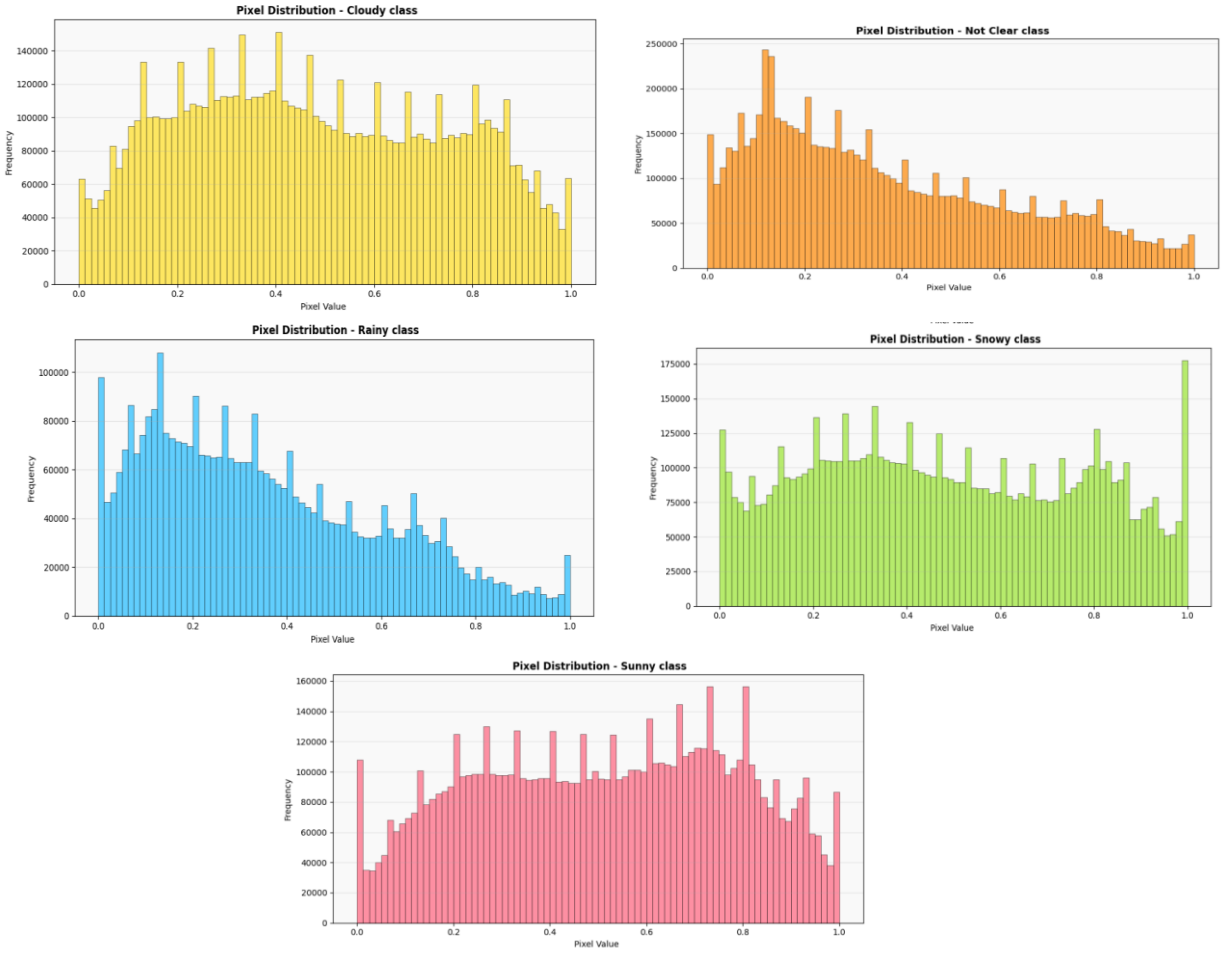


Figure 6:Pixel distribution

4- Experiments and Results

For all models we split the data into 70% train, 15% validation, and 15% test using stratified sampling to keep class ratios.

4.1-Baseline Model (k-NN)

k-NN was implemented as a distance-based classifier: each image was resized to a fixed size, then flattened into a 1D feature vector. For a test sample, we computed the Euclidean distance to all training vectors, selected the k closest neighbors (k = 1 and k = 3), and assigned the label by majority vote.

4.2- Proposed Models

SVM :We trained a Support Vector Machine classifier with an RBF kernel. SVM works by finding the hyperplane that best separates different classes with the maximum margin, and the RBF kernel allows it to capture non-linear relationships in high-dimensional feature spaces. The original image features were flattened and standardized to ensure consistent scaling, which is important for SVM performance. Principal Component Analysis was applied to reduce the dimensionality to 200 components, improving computational efficiency and helping the model generalize

better. Hyperparameter tuning included the regularization parameter C = [0.1, 1, 10, 100], which controls the trade-off between margin maximization and classification error, gamma='scale', which automatically adjusts the kernel coefficient based on the feature space, and class_weight='balanced', which compensates for class imbalance by assigning higher importance to minority classes.

CNN: We trained a CNN using transfer learning with EfficientNet-B0 (include_top=False) on .npy image folders, using batch size = 32 and the Adam optimizer. To address class imbalance, we applied class weights computed from the training data. Overfitting was reduced using data augmentation, Dropout = 0.4, EarlyStopping (patience = 4), and ReduceLROnPlateau (patience = 2, factor = 0.3). Training was performed in two stages: first training the classification head with all EfficientNet layers frozen for 12 epochs, then fine-tuning the last 15 layers for 10 epochs with a reduced learning rate (0.1×).

4.3- Hyperparameter Tuning

Hyperparameter tuning was performed for each model by focusing on the most influential parameter. For SVM, the regularization parameter C was tuned because it directly controls the trade-off between model complexity and generalization, making it the most important hyperparameter for performance. For CNN, the learning rate was tuned since it has the strongest effect on training stability, convergence speed, and final accuracy. Other parameters were kept fixed to ensure a fair comparison and stable training, while testing multiple values allowed us to select the best-performing configuration.

4.4- Results Presentation

The performance of all models was evaluated using accuracy, macro-averaged F1-score, and class-wise precision and recall. In addition, confusion matrices were used to analyze classification behavior across different classes. The results are presented below using tables for clarity and ease of comparison.

Table 1 : Hyperparameter Tuning (CNN)

Learning Rate	Val Accuracy	Val Macro F1
1e-3	0.5570	0.4520
3e-4	0.4631	0.3987
1e-4	0.3356	0.2577
3e-5	0.2617	0.2290

Model selection was based on validation macro F1-score due to class imbalance its when learning rate = **1e-3**

Table 2 : Hyperparameter Tuning (SVM)

Model	Train Accuracy	Validation Accuracy	Test Accuracy
SVM (C = 0.1)	0.3896	0.2905	0.3020

SVM (C = 1)	0.7302	0.4324	0.4564
SVM (C = 10)	0.9913	0.5878	0.5906
SVM (C = 100)	0.9971	0.5811	0.5973

Model selection was based on Test Accuracy its when C= 10.

4.5-Confusion Matrices

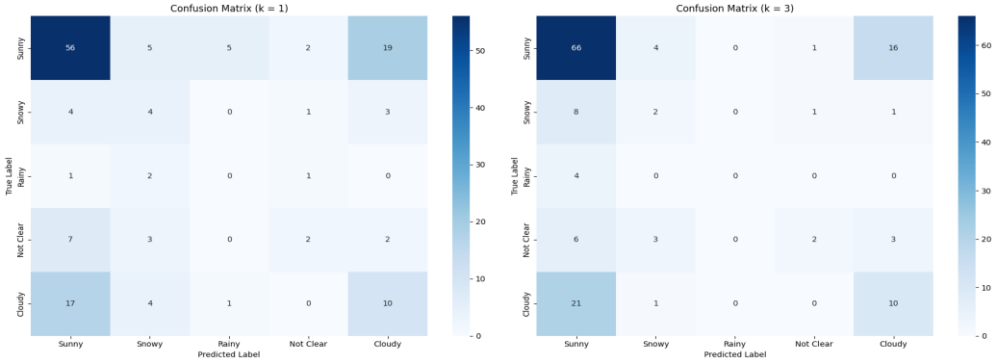


Figure 7 : Confusion Matrix (K=1)(K=3)

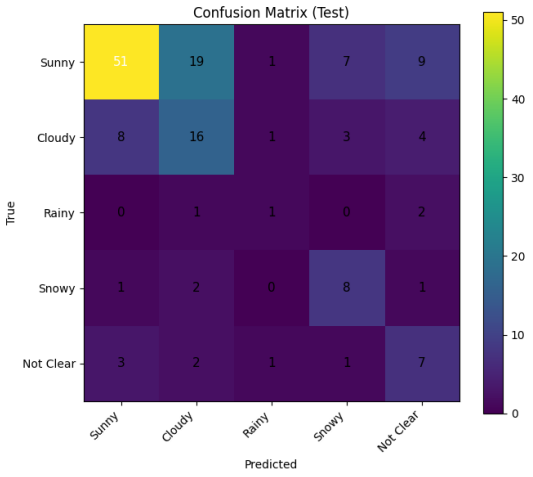


Figure 8 : Confusion Matrix SVM

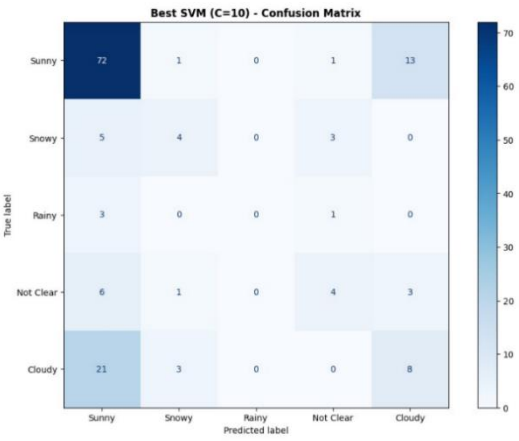


Figure 9 : Confusion Matrix (CNN)

In CNN: Loss vs Epoch & Accuracy vs Epoch Early stopping without Overfitting:

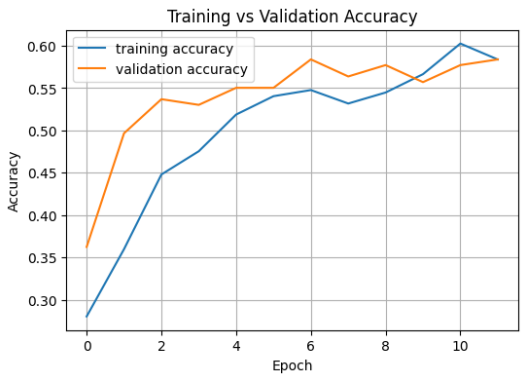


Figure 10 : Accuracy vs Epoch

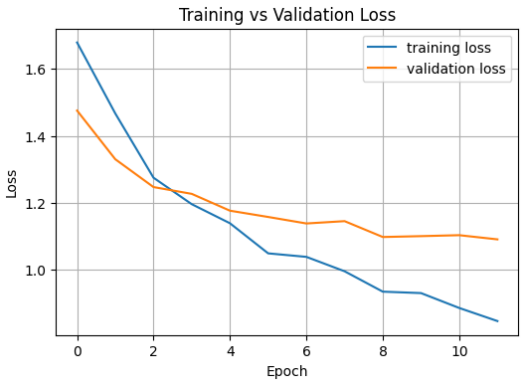


Figure 11 : Loss vs Epoch

4.6-Interpretation and Comparison

Table 3 : Final Results Accuracy Comparison

Model	Train Accuracy	Validation Accuracy	Test Accuracy
k-NN (k = 1)	0.9971	0.4797	0.4832
k-NN (k = 3)	0.7388	0.5811	0.5369
SVM	0.9913	0.5878	0.5906
CNN	0.6260	0.5570	0.5570

Table 4 : Final Results Precision & Recall & F1-score Comparison

Model	Precision (Macro)	Recall (Macro)	F1-score (Macro)
k-NN (k = 1)	0.3017	0.2865	0.2842
k-NN (k = 3)	0.3324	0.2761	0.2828
SVM	0.38	0.34	0.35
CNN	0.4314	0.4451	0.4294

The k-NN and SVM models show clear signs of overfitting, achieving very high training accuracy but much lower validation and test performance, which indicates weak generalization. In contrast, the CNN demonstrates better feature learning and more stable performance across training, validation, and test sets. Although the CNN's accuracy is slightly lower than SVM, its higher macro F1-score shows more balanced classification across classes under imbalance. Notably, the CNN was the only model able to correctly classify the *Rainy* class, despite its very small number of samples, while k-NN and SVM struggled to recognize this minority class.

5- Performance Analysis

	precision	recall	f1-score	support
Sunny	0.8095	0.5862	0.6800	87
Cloudy	0.4000	0.5000	0.4444	32
Rainy	0.2500	0.2500	0.2500	4
Snowy	0.4211	0.6667	0.5161	12
Not Clear	0.3043	0.5000	0.3784	14
accuracy			0.5570	149
macro avg	0.4370	0.5006	0.4538	149
weighted avg	0.6278	0.5570	0.5763	149

Figure 12 : Classification Report



We analyzed CNN errors on the test set (n=149) using the confusion matrix and the per-class precision/recall/F1 report, and by reviewing misclassified test samples. The errors were not random; they were concentrated in visually similar classes. The most common confusion occurred when Sunny images were predicted as Cloudy (19), Not Clear (9), or Snowy (7), which explains the low Sunny recall (0.55) and suggests that sunny cues (brightness and clear sky) were weak or partially hidden in many images. The Cloudy class showed moderate recall (0.47) but low precision (0.32), meaning the model predicts Cloudy too often and uses it as a "fallback" label for borderline cases. The Rainy class was the most difficult due to extremely small support (only 4 test images), leading to unstable performance (recall 0.25) and confusion mainly with Not Clear and Cloudy. In contrast, Snowy achieved the highest recall among minority classes (0.75), indicating that snow provides stronger visual cues that are easier to detect. By inspecting misclassified test examples, we observed that many errors occurred in images with weak or ambiguous weather cues, such as low brightness, partially occluded skies (e.g., buildings or people), or mixed visual conditions that make weather identification difficult from a single image. Overall, the main error patterns are driven by visual overlap between classes (especially Sunny–Cloudy–Not Clear) and dataset issues, including class imbalance and likely label ambiguity or noise

6- Conclusions and Discussion

This work demonstrates that deep learning approaches are more suitable for weather classification from images than traditional machine learning models. While k-NN and SVM achieved high training accuracy, their performance dropped significantly on unseen data, indicating limited generalization. In contrast, the CNN showed stronger learning capability and more stable performance across training, validation, and test sets, making it the most reliable model for this task.

Despite these results, several limitations affected the overall performance. The dataset suffers from severe class imbalance, particularly for the *Rainy* class, which limits the model's ability to learn robust representations for this category. In addition, some images were incorrectly or ambiguously labeled, as students who performed the annotation made mistakes in labeling certain samples. This labeling noise negatively impacted both model training and evaluation. Furthermore, some weather conditions

are visually subtle or overlapping, making them difficult to distinguish from a single image.

Future work should focus on collecting a larger dataset and balancing the classes, especially by adding more *Rainy* images. We also need to manually verify and correct labels to ensure each image is assigned to the correct class, since labeling errors can reduce performance. In addition, we can apply oversampling or balanced batch sampling for minority classes and experiment with focal loss to better handle rare and difficult examples. Finally, incorporating contextual information (e.g., time or location) and using Grad-CAM to interpret model decisions may further improve robustness and error analysis.

THE END 😊